



CIÊNCIA DA
COMPUTAÇÃO
GRANDE FLORIANÓPOLIS



Curso de Ciência da Computação

Programação Orientada a Objetos – POO

Profa. Fernanda dos Santos Cunha

Programas exemplo retirados de **Java: como programar**
Deitel, P.; Deitel, H. - São Paulo: Pearson Education do Brasil, 2017.

Orientação a Objetos

Modificadores de acesso

- O **encapsulamento** serve para ocultar os dados, evitando o acesso direto a membros (proteção do código) e permitindo a modificação interna de uma classe sem alterar a sua funcionalidade e o modo como é acessada.
 - Utilizado em Java através dos modificadores de acesso.

public	É visível em qualquer lugar.
protected	Só é visível na mesma classe e em suas subclasses.
package	Default. Só é visível em classes do mesmo pacote. Não existe modificador com este nome. A ausência de modificador o torna package.
private	Só é visível dentro da mesma classe.

Orientação a Objetos

Sobrecarga e Sobrescrita de métodos



- A **sobrecarga de métodos** (*overload*) permite a existência de métodos **de** mesmo nome, mas com assinaturas diferentes, ex. *diferentes* conjuntos de parâmetros (determinados pelo número, tipos e ordem dos parâmetros).
 - Quando um método sobrecarregado é chamado, o compilador Java seleciona o método adequado examinando o número, os tipos e a ordem dos argumentos na chamada.
 - A sobrecarga de métodos é comumente utilizada para criar vários métodos com o *mesmo* nome que realizam as *mesmas* tarefas, ou tarefas *semelhantes*, mas sobre tipos *diferentes* ou números *diferentes* de argumentos.
 - Inclusive os construtores podem ser sobrecarregados.

Orientação a Objetos

Sobrecarga e Sobrescrita de métodos

- A **sobrescrita de métodos** (*override*) ocorre quando se cria um novo método na classe filha (subclasse) contendo a mesma assinatura e mesmo tipo **de** retorno do método **sobrescrito existente na classe mãe** (superclasse).
 - Claramente isso refere-se a **Herança** (relacionamento Generalização)

Sobrecarga e Sobreescrita de métodos

- Exemplo classe Time com construtores sobrecarregados

```
Time.java x TimeTest.java x
1 // Figura 8.5: Time.java
2 // declaração da classe Time com construtores sobrecarregados.
3 public class Time {
4     private int hour; // 0 - 23
5     private int minute; // 0 - 59
6     private int second; // 0 - 59
7     // construtor sem argumento Time: inicializa cada variável de instância para zero
8     public Time() {
9         this( hour: 0, minute: 0, second: 0); // invoca o construtor com três argumentos
10    }
11    // Construtor Time: hora fornecida, minuto e segundo padronizados para 0
12    public Time(int hour) {
13        this(hour, minute: 0, second: 0); // invoca o construtor com três argumentos
14    }
15    // Construtor Time: hora e minuto fornecidos, segundo padronizado para 0
16    public Time(int hour, int minute) {
17        this(hour, minute, second: 0); // invoca o construtor com três argumentos
18    }
19    // Construtor Time: hour, minute e second fornecidos
20    public Time(int hour, int minute, int second){
21        if (hour < 0 || hour >= 24)
22            throw new IllegalArgumentException("hour must be 0-23");
23        if (minute < 0 || minute >= 60)
24            throw new IllegalArgumentException("minute must be 0-59");
25        if (second < 0 || second >= 60)
26            throw new IllegalArgumentException("second must be 0-59");
27        this.hour = hour;
28        this.minute = minute;
29        this.second = second;
30    }
31 }
```


Sobrecarga e Sobrescrita de métodos



- Exemplo classe Time com construtores sobrecarregados

```
Time.java x TimeTest.java x
32 // Construtor Time: outro objeto Time fornecido
33 @ public Time(Time time) {
34     // invoca o construtor com três argumentos
35     this(time.getHour(), time.getMinute(), time.getSecond());
36 }
37 // Métodos set
38 // Configura um novo valor de tempo usando hora universal;
39 // valida os dados
40 public void setTime(int hour, int minute, int second) {
41     if (hour < 0 || hour >= 24)
42         throw new IllegalArgumentException("hour must be 0-23");
43     if (minute < 0 || minute >= 60)
44         throw new IllegalArgumentException("minute must be 0-59");
45     if (second < 0 || second >= 60)
46         throw new IllegalArgumentException("second must be 0-59");
47     this.hour = hour;
48     this.minute = minute;
49     this.second = second;
50 }
51 // valida e configura a hora
52 public void setHour(int hour) {
53     if (hour < 0 || hour >= 24)
54         throw new IllegalArgumentException("hour must be 0-23");
55     this.hour = hour;
56 }
57 // valida e configura os minutos
58 public void setMinute(int minute) {
59     if (minute < 0 || minute >= 60)
60         throw new IllegalArgumentException("minute must be 0-59");
61     this.minute = minute;
62 }
```

Sobrecarga e Sobrescrita de métodos



- Exemplo classe Time com construtores sobrecarregados

```
Time.java x TimeTest.java x
63 // valida e configura os segundos
64 public void setSecond(int second) {
65     if (second < 0 || second >= 60)
66         throw new IllegalArgumentException("second must be 0-59");
67     this.second = second;
68 }
69 // Métodos get
70 // obtém valor da hora
71 public int getHour() {
72     return hour;
73 }
74 // obtém valor dos minutos
75 public int getMinute() {
76     return minute;
77 }
78 // obtém valor dos segundos
79 public int getSecond() {
80     return second;
81 }
82 // converte em String no formato de data/hora universal (HH:MM:SS)
83 public String toUniversalString() {
84     return String.format(
85         "%02d:%02d:%02d", getHour(), getMinute(), getSecond());
86 }
87 // converte em String no formato padrão de data/hora (H:MM:SS AM ou PM)
88 public String toString() {
89     return String.format("%d:%02d:%02d %s",
90         ((getHour() == 0 || getHour() == 12) ? 12 : getHour() % 12),
91         getMinute(), getSecond(), (getHour() < 12 ? "AM" : "PM"));
92 }
93 } // fim da classe Time
```

Sobrecarga e Sobrescrita de métodos



- Exemplo classe Time com construtores sobrecarregados

```
Time.java x TimeTest.java x
1 // Figura 8.6: TimeTest.java
2 // Construtores sobrecarregados utilizados para inicializar objetos Time.
3 public class TimeTest {
4     public static void main(String[] args) {
5         Time t1 = new Time(); // 00:00:00
6         Time t2 = new Time( hour: 2); // 02:00:00
7         Time t3 = new Time( hour: 21, minute: 34); // 21:34:00
8         Time t4 = new Time( hour: 12, minute: 25, second: 42); // 12:25:42
9         Time t5 = new Time(t4); // 12:25:42
10
11         System.out.println("Constructed with:");
12         displayTime( header: "t1: all default arguments", t1);
13         displayTime( header: "t2: hour specified; default minute and second", t2);
14         displayTime( header: "t3: hour and minute specified; default second", t3);
15         displayTime( header: "t4: hour, minute and second specified", t4);
16         displayTime( header: "t5: Time2 object t4 specified", t5);
17
18         // tenta inicializar t6 com valores inválidos
19         try {
20             Time t6 = new Time( hour: 27, minute: 74, second: 99); // valores inválidos
21         } catch (IllegalArgumentException e) {
22             System.out.printf("\nException while initializing t6: %s\n", e.getMessage());
23         }
24
25         // exibe um objeto Time nos formatos de 24 horas e 12 horas
26 @ private static void displayTime(String header, Time t) {
27     System.out.printf("%s\n %s\n %s\n",
28         header, t.toUniversalString(), t.toString());
29 }
30 } // fim da classe
31
```

Constructed with:
t1: all default arguments
00:00:00
12:00:00 AM
t2: hour specified; default minute and second
02:00:00
2:00:00 AM
t3: hour and minute specified; default second
21:34:00
9:34:00 PM
t4: hour, minute and second specified
12:25:42
12:25:42 PM
t5: Time2 object t4 specified
12:25:42
12:25:42 PM
Exception while initializing t6: hour must be 0-23

Orientação a Objetos

- Analisando as classes abaixo vê-se uma duplicação de código... **Que tal repensá-las?**

Pessoa
- cpf :String - dataNascimento :Calendar - nome :String
+ getCpf() :String + getDataNascimento() :Calendar + getIdade() :int + getNome() :String + setCpf(cpf :String) :void + setDataNascimento(data :Calendar) :void + setNome(nome :String) :void

Aluno
- cpf :String - dataNascimento :Calendar - nome :String - matricula :String - curso :String
+ setNome(nome :String) :void + getNome() :String + setDataNascimento(data :Calendar) :void + getDataNascimento() :Calendar + getIdade() :int + getCpf() :String + setCpf(cpf :String) :void + getMatricula() :String + setMatricula(newVal :String) :void + getCurso() :String + setCurso(newVal :String) :void

Orientação a Objetos

- Melhor assim!

Pessoa
- cpf :String - dataNascimento :Calendar - nome :String
+ setNome(String) :void + getNome() :String + setDataNascimento(Calendar) :void + getDataNascimento() :Calendar + getIdade() :int + getCpf() :String + setCpf(String) :void

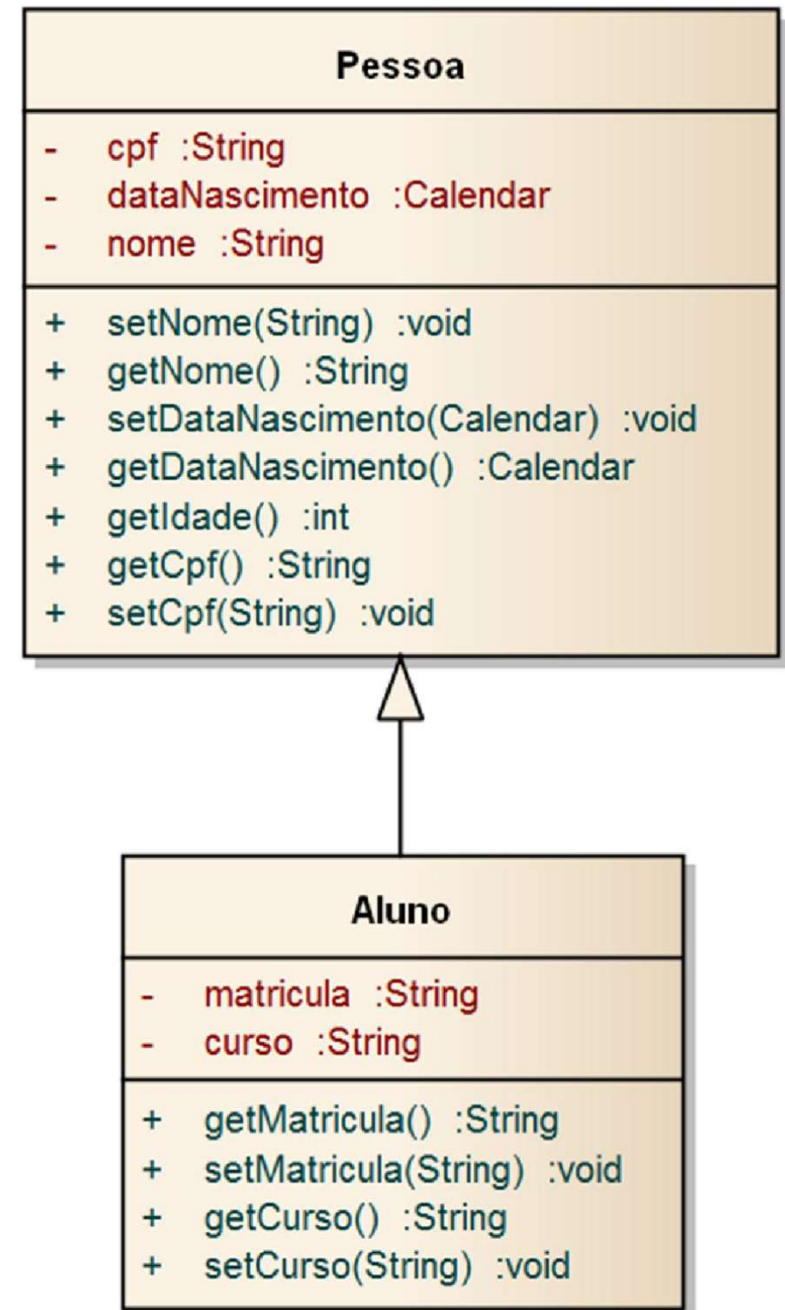
Aluno
- matricula :String - curso :String
+ getMatricula() :String + setMatricula(String) :void + getCurso() :String + setCurso(String) :void

=> Mas ainda falta algo... Como sinalizar que **um aluno é uma pessoa**? Deve haver uma ligação entre as classes!

Orientação a Objetos

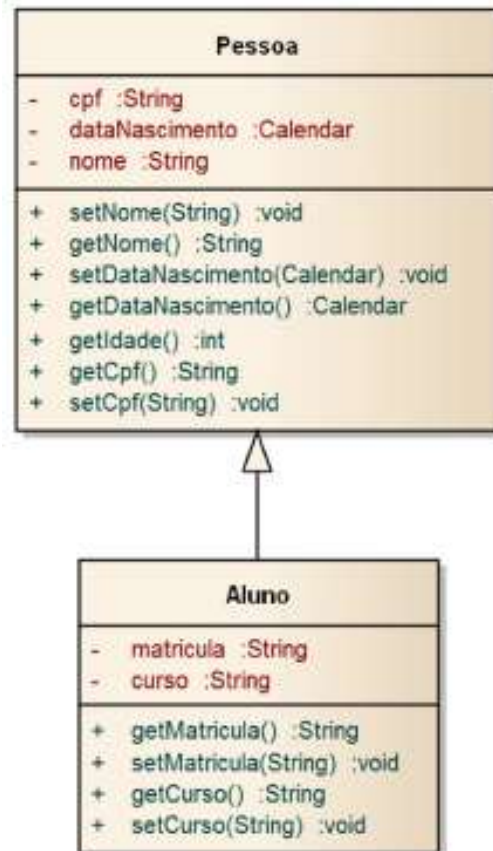
Herança

- Também chamada Generalização/Especialização, relacionamento “é um/é uma”
- Pessoa é a **superclasse** (mais genérica) e Aluno é a **subclasse** (mais especializada)
- O sentido do relacionamento é da classe **mais especializada** para a **mais genérica**
- Logo, a subclasse **Aluno herda todos** os atributos e operações da superclasse **Pessoa**



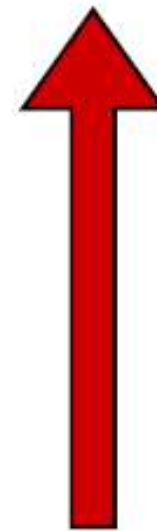
Orientação a Objetos

Herança



Maior Abstração
reusabilidade mais alta

Sentido da
generalização



Sentido da
especialização

Menor Abstração
reusabilidade mais baixa

Orientação a Objetos

Herança

- Cada objeto (instância) da subclasse é também uma instância indireta da superclasse
- Se esta classe base for derivada de outra classe, o processo se repete recursivamente até classe não derivada seja alcançada
 - Processo essencial para consistência do objeto recém criado

Superclasse	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

Figura 9.1 | Exemplos de herança.

Orientação a Objetos

Herança



- Como declarar uma classe derivada:

```
public SubClasse extends SuperClasse {  
    // declarar os atributos  
    // e métodos específicos desta subclasse  
}
```

- O construtor da subclasse deverá acionar o construtor da sua classe base via palavra reservada **super**:

```
public ClasseDerivada (<parâmetros>) {  
    super (<parâmetros da superclasse>);  
    ...  
}
```

- Operações podem ser **sobrescritas** (`@Override`).
 - Novo código conforme a especialização feita.
 - Ex.: método `toString`

Orientação a Objetos

Herança

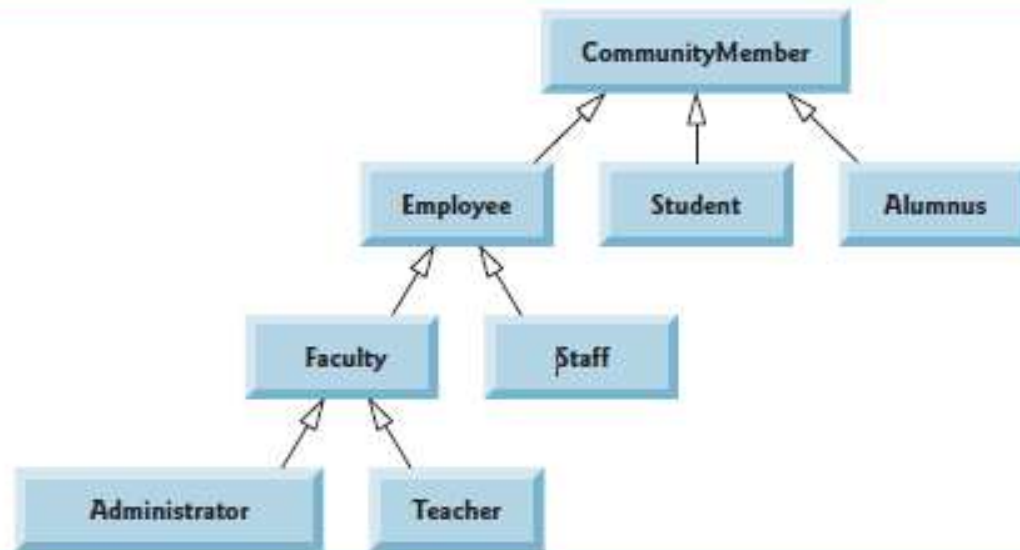


Figura 9.2 | Diagrama de classes UML da hierarquia de herança para CommunityMembers universitários.

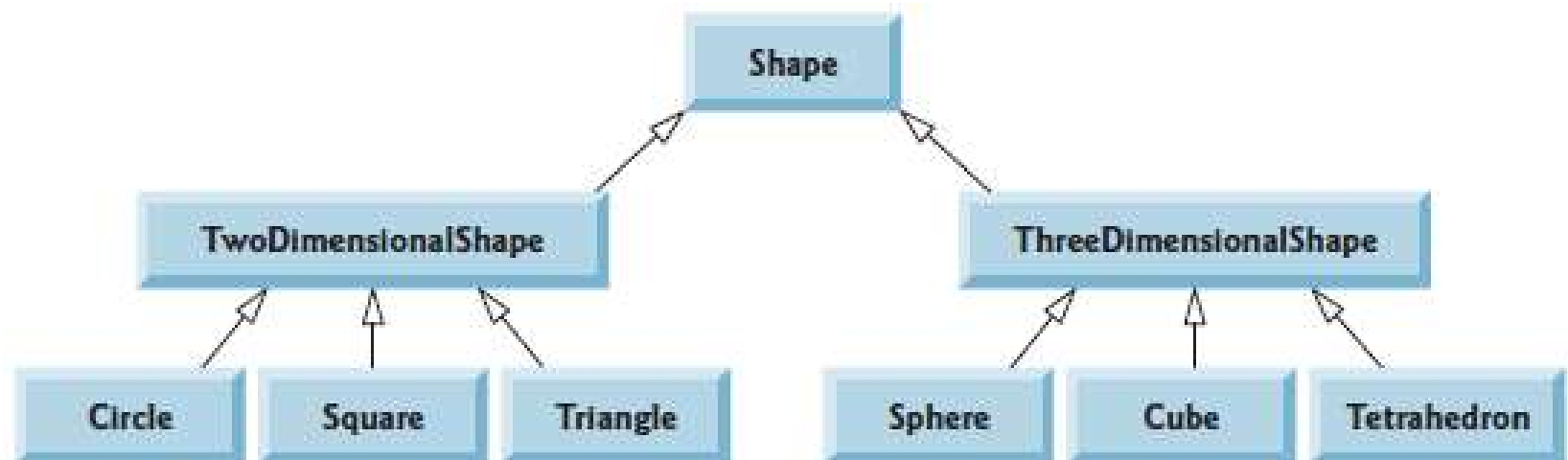


Figura 9.3 | Diagrama de classes UML da hierarquia de herança para Shapes.

Orientação a Objetos

Herança



- Avalie a quantidade de níveis de uma herança – mais do que 3 podem indicar problemas de modelagem
- Pense em facilitar futuras mudanças na hierarquia de classes
- Não misture domínio do negócio (`Professor`, `Aluno`) com classes de implementação (`ArrayList`)
 - Classes de domínio **usam** classes de implementação
- Não comece a modelagem do projeto já pensando em herança
 - Abstraia corretamente o problema
 - Analise as semelhanças semânticas

Herança em Java

Esta é a superclasse

```
CommissionEmployee.java x BasePlusCommissionEmployee.java x BasePlusCommissionEmployeeTest.java x
1 public class CommissionEmployee {
2     private final String firstName;
3     private final String lastName;
4     private final String socialSecurityNumber;
5     private double grossSales; // vendas brutas semanais
6     private double commissionRate; // porcentagem da comissão
7     public CommissionEmployee(String firstName, String lastName, String socialSecurityNumber,
8                               double grossSales, double commissionRate) {
9         // se grossSales é inválido, lança uma exceção
10        if (grossSales < 0.0)
11            throw new IllegalArgumentException("Gross sales must be >= 0.0");
12
13        // se commissionRate é inválido, lança uma exceção
14        if (commissionRate <= 0.0 || commissionRate >= 1.0)
15            throw new IllegalArgumentException("Commission rate must be > 0.0 and < 1.0");
16
17        this.firstName = firstName;
18        this.lastName = lastName;
19        this.socialSecurityNumber = socialSecurityNumber;
20        this.grossSales = grossSales;
21        this.commissionRate = commissionRate;
22    }
23    public String getFirstName() {
24        return firstName;
25    }
26    public String getLastName() {
27        return lastName;
28    }
29    public String getSocialSecurityNumber() {
30        return socialSecurityNumber;
31    }
```

Herança em Java

Esta é a superclasse

```
CommissionEmployee.java x BasePlusCommissionEmployee.java x BasePlusCommissionEmployeeTest.java x
32 public double getGrossSales() {
33     return grossSales;
34 }
35 public void setGrossSales(double grossSales) {
36     // se grossSales é inválido, lança uma exceção
37     if (grossSales < 0.0)
38         throw new IllegalArgumentException("Gross sales must be >= 0.0");
39     this.grossSales = grossSales;
40 }
41 public double getCommissionRate() {
42     return commissionRate;
43 }
44 public void setCommissionRate(double commissionRate) {
45     if (commissionRate <= 0.0 || commissionRate >= 1.0)
46         throw new IllegalArgumentException("Commission rate must be > 0.0 and < 1.0");
47     this.commissionRate = commissionRate;
48 }
49 // calcula os lucros
50 public double earnings() {
51     return getCommissionRate() * getGrossSales();
52 }
53 @Override //retorna a representação String do objeto CommissionEmployee
54 public String toString() {
55     return String.format("%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
56         "commission employee", getFirstName(), getLastName(),
57         "social security number", getSocialSecurityNumber(),
58         "gross sales", getGrossSales(),
59         "commission rate", getCommissionRate());
60 }
61 }
62 }
```

Herança em Java

Esta é a subclasse – veja o `extends`

```
CommissionEmployee.java x BasePlusCommissionEmployee.java x BasePlusCommissionEmployeeTest.java x
1 // Figura 9.11: BasePlusCommissionEmployee.java
2 // A classe BasePlusCommissionEmployee é herdada de CommissionEmployee e acessa os dados private da
3 // superclasse via métodos public herdados.
4 public class BasePlusCommissionEmployee extends CommissionEmployee {
5     private double baseSalary; // salário-base por semana
6
7     public BasePlusCommissionEmployee(String firstName, String lastName, String socialSecurityNumber,
8         double grossSales, double commissionRate, double baseSalary)
9     {
10         // chamada construtor superclasse
11         super(firstName, lastName, socialSecurityNumber, grossSales, commissionRate);
12         // se baseSalary é inválido, lança uma exceção
13         if (baseSalary < 0.0)
14             throw new IllegalArgumentException(
15                 "Base salary must be >= 0.0");
16         this.baseSalary = baseSalary;
17     }
18     public double getBaseSalary() { return baseSalary; }
21     public void setBaseSalary(double baseSalary) {...} // código igual linhas 12-15 do construtor
28     // calcula os lucros
29     @Override
30     public double earnings() {
31         return getBaseSalary() * super.earnings();
32     }
33     // retorna a representação de String de BasePlusCommissionEmployee
34     @Override
35     public String toString() {
36         return String.format("%s %s%n%s: %.2f", "base-salaried",
37             super.toString(), "base salary", getBaseSalary());
38     }
39 }
```


Herança em Java

Esta é a classe teste



```
CommissionEmployee.java x BasePlusCommissionEmployee.java x BasePlusCommissionEmployeeTest.java x
1 public class BasePlusCommissionEmployeeTest {
2     public static void main(String[] args){
3         // instancia o objeto BasePlusCommissionEmployee
4         BasePlusCommissionEmployee employee =
5             new BasePlusCommissionEmployee( firstName: "Bob",  lastName: "Lewis",  socialSecurityNumber: "333-33-3333",
6                 grossSales: 5000,  commissionRate: .04,  baseSalary: 300);
7         // obtém os dados do empregado comissionado com salário-base
8         System.out.println("Employee information obtained by get methods:\n");
9         System.out.printf("%s %s\n", "First name is",employee.getFirstName());
10        System.out.printf("%s %s\n", "Last name is",employee.getLastName());
11        System.out.printf("%s %s\n", "Social security number is",employee.getSocialSecurityNumber());
12        System.out.printf("%s %.2f\n", "Gross sales is",employee.getGrossSales());
13        System.out.printf("%s %.2f\n", "Commission rate is",employee.getCommissionRate());
14        System.out.printf("%s %.2f\n", "Base salary is",employee.getBaseSalary());
15        employee.setBaseSalary(1000);
16        System.out.printf("\n%s:\n%s\n", "Updated employee information obtained by toString",employee.toString());
17    } // fim de main
18
19 }
```

Run: BasePlusCommissionEmployeeTest x

Base salary is 300,00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000,00
commission rate: 0,04
base salary: 1000,00

Process finished with exit code 0

Java

Classe Object



- Todas as classes em Java herdam direta ou indiretamente da classe `Object` (pacote `java.lang`), então seus 11 métodos (alguns sobrecarregados) são herdados por todas as outras classes.
- Abaixo e no próximo slide apresenta-se uma tabela resumo dos métodos de `Object` (Figura 9.12 do Deitel).

Método	Descrição
<code>equals</code>	Esse método compara dois objetos quanto à igualdade e retorna <code>true</code> se eles forem iguais; caso contrário, retorna <code>false</code> . O método aceita qualquer <code>Object</code> como um argumento. Quando os objetos de uma classe particular precisarem ser comparados em relação à igualdade, a classe deve sobrescrever o método <code>equals</code> a fim de comparar o <i>conteúdo</i> dos dois objetos. Para os requisitos da implementação desse método (que incluem também o método de sobrescrição <code>hashCode</code>), consulte a documentação dele em <code>docs.oracle.com/javase/7/docs/api/java/lang/Object.html#equals(java.lang.Object)</code> . A implementação <code>equals</code> padrão utiliza o operador <code>==</code> para determinar se duas referências <i>referenciam o mesmo objeto</i> na memória. A Seção 14.3.3 demonstra o método <code>equals</code> da classe <code>String</code> e diferencia entre comparar objetos <code>String</code> com <code>==</code> e com <code>equals</code> .

Java

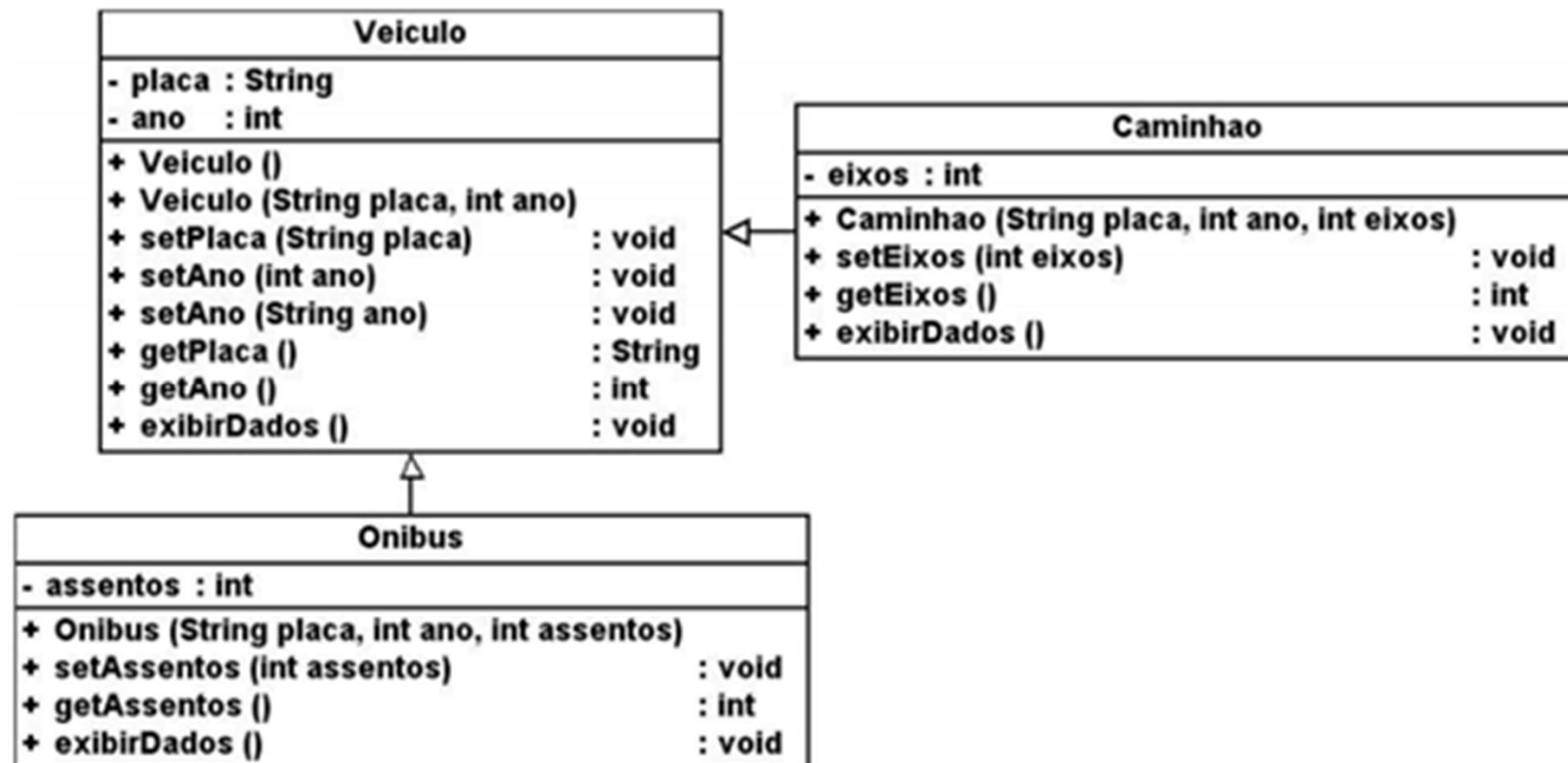
Classe Object



Método	Descrição
<code>hashCode</code>	Hashcodes são valores <code>int</code> utilizados para armazenamento e recuperação de alta velocidade das informações mantidas em uma estrutura de dados que é conhecida como tabela de hash (veja a Seção 16.11). Esse método também é chamado como parte da implementação padrão do método <code>toString</code> de <code>Object</code> .
<code>toString</code>	Esse método (introduzido na Seção 9.4.1) retorna uma representação <code>String</code> de um objeto. A implementação padrão desse método retorna o nome do pacote e o nome da classe do objeto tipicamente seguido por uma representação hexadecimal do valor retornado pelo método <code>hashCode</code> do objeto.
<code>wait</code> , <code>notify</code> , <code>notifyAll</code>	Os métodos <code>notify</code> e <code>notifyAll</code> e as três versões sobrecarregadas de <code>wait</code> estão relacionados com <code>multithreading</code> , o que será discutido no Capítulo 23.
<code>getClass</code>	Todo objeto no Java conhece seu próprio tipo em tempo de execução. O método <code>getClass</code> (utilizado na Seção 10.5 e Seção 12.5) retorna um objeto de classe <code>Class</code> (pacote <code>java.lang</code>) que contém as informações sobre o tipo de objeto, como seu nome de classe (retornado pelo método <code>Class.getName</code>).
<code>finalize</code>	O método <code>protected</code> é chamado pelo coletor de lixo para realizar limpeza de terminação sobre um objeto um pouco antes desse coletor reivindicar a memória do objeto. Lembre-se da Seção 8.10 que não fica claro se, ou quando, <code>finalize</code> será chamado. Por essa razão, a maioria dos programadores deve evitar o método <code>finalize</code> .
<code>clone</code>	Esse método <code>protected</code> , que não aceita nenhum argumento e retorna uma referência <code>Object</code> , faz uma cópia do objeto em que é chamado. A implementação padrão realiza a chamada cópia superficial — os valores da variável de instância em um objeto são copiados em outro objeto do mesmo tipo. Para tipos por referência, apenas as referências são copiadas. Uma típica implementação do método <code>clone</code> sobrescrito realizaria uma cópia em profundidade que cria um novo objeto para cada variável de instância de tipo por referência. <i>É difícil implementar <code>clone</code> corretamente. Por essa razão, seu uso não é recomendado.</i> Alguns especialistas da indústria sugerem que, em vez disso, deve ser usada a serialização de objetos.

EXERCÍCIOS

1. Crie as classes Pessoa (atributos nome e cpf), Funcionario (subclasse de Pessoa, atributo salário) e Gerente (subclasse de Funcionario, atributos área e gratificação). Teste-as.
2. Implemente as classes do diagrama abaixo.



LEITURA PARA CASA



Schildt, Herbert. Java para iniciantes.

Cap 6 – Verificação minuciosa dos métodos e classes

Cap 7 – Herança